ChatGPT Clone API Documentation

This document explains the backend request, response, and service logic for the chat APIs in the ChatGPT clone project. The examples describe the expected server-side behavior while leaving room for different implementation styles.

Base URL: `http://localhost:3777`
API prefix: `/api`
Chat route prefix: `/api/chat`

GET `/api/chat/conversations`

Fetch saved chat messages for the frontend chat history.

What This API Does

This endpoint reads stored messages from the database and returns them to the frontend. It does not create a new message and does not call the AI provider.

The frontend can use this endpoint when:

- The chat page first loads.
- The user refreshes the page.
- The app needs to rebuild the visible chat history from the database.

Request

Item	Value
Method	`GET`
Path	`/api/chat/conversations`
Query params	None
Body	None
Auth	None in this project

Postman Testing

To test this API in Postman:

1. Create a new request.
2. Set the method to `GET`.
3. Enter this URL: `http://localhost:3777/api/chat/conversations`.
4. Do not add a request body.
5. Click **Send**.

Server-Side Logic

Generic service flow:

1. Receive a `GET` request from the frontend.
2. Query the `conversations` table.
3. Select message fields: `id`, `role`, `content`, `token_count`, and `created_at`.
4. Read the latest 100 rows by ordering with `id DESC` and using `LIMIT 100`.
5. Reverse the result before sending it to the frontend.
6. Convert database field names to API field names:
   - `token_count` becomes `tokenCount`.
   - `created_at` becomes `createdAt`.
7. Return the messages inside `data.conversations`.

Why the result is reversed:

- The database query gets the newest messages first.
- Chat UIs usually display older messages first and newer messages last.
- Reversing the latest 100 rows gives the frontend chronological order for that selected batch.

Success Response

Status: `200 OK`

Field	Type	Description
`success`	`boolean`	`true` when messages are fetched successfully.
`message`	`string`	Success message.
`data`	`object`	Response payload wrapper.
`data.conversations`	`array`	List of conversation message objects.

Each object inside `data.conversations`:

Field	Type	Description
`id`	`number`	Message id from the database.

Field	Type	Description
`role`	`string`	Either `"user"` or `"assistant"`.
`content`	`string`	Message text.
`tokenCount`	`number`	Token usage from the database. User messages usually return `0`.
`createdAt`	`string`	Time the message was created.

Example:

```
{
  "success": true,
  "message": "Conversations fetched successfully.",
  "data": {
    "conversations": [
      {
        "id": 1,
        "role": "user",
        "content": "Hello",
        "tokenCount": 0,
        "createdAt": "2026-05-10T10:00:00.000Z"
      },
      {
        "id": 2,
        "role": "assistant",
        "content": "Hi! How can I help with your code today?",
        "tokenCount": 42,
        "createdAt": "2026-05-10T10:00:05.000Z"
      }
    ]
  }
}
```

If there are no saved messages, return an empty array:

```
{
  "success": true,
  "message": "Conversations fetched successfully.",
  "data": {
    "conversations": []
  }
}
```

Error Response

Status: `500 Internal Server Error`

```
{
  "status": false,
  "message": "Something went wrong try again later"
}
```

POST `/api/chat/conversations`

Save a user question, generate an assistant answer, save the assistant answer, and return both saved rows.

What This API Does

This endpoint handles one complete user-to-assistant chat turn.

When the frontend sends a question, the backend should:

- Validate the question.
- Store the user's question.
- Ask the AI provider for an answer.
- Store the assistant's answer.
- Return both saved database rows.

This API is not idempotent. Every successful request creates new database rows. If the frontend sends the same question twice, the database should store two separate user messages and two separate assistant responses.

Request

Item	Value
Method	`POST`
Path	`/api/chat/conversations`
Headers	`Content-Type: application/json`
Body	JSON object
Auth	None in this project

Body fields:

Field	Type	Required	Rules
`question`	`string`	Yes	Must not be empty after trimming spaces. Maximum length is `65535` characters.

Example:

```
{
  "question": "What is a closure in JavaScript?"
}
```

Postman Testing

To test this API in Postman:

1. Create a new request.
2. Set the method to `POST`.
3. Enter this URL: `http://localhost:3777/api/chat/conversations`.
4. Open the **Body** tab.
5. Select **raw**.
6. Select **JSON** from the body type dropdown.
7. Add this JSON body:

```
{
  "question": "What is a closure in JavaScript?"
}
```

8. Click **Send**.

Server-Side Logic

Generic service flow:

1. Receive the request body from the frontend.
2. Validate `question`.
   - It must exist.
   - It must be a non-empty string after trimming spaces.
   - It must not be longer than `65535` characters.
3. Normalize the question by trimming extra spaces from the beginning and end.
4. Load a small amount of previous conversation history from the database.

    - Read the latest 5 rows.
    - Reverse them into chronological order.
    - This history is used only as AI context.
5. Insert the user's new message into the database.
    - `role` should be `"user"`.
    - `content` should be the trimmed question.
    - `token_count` can use the database default value of `0`.
6. Format the previous history for the AI provider.
    - User rows stay as user messages.
    - Assistant rows may need to be converted to the provider's assistant/model role.
7. Send the previous history and the new question to the AI provider.
8. Use a system instruction that keeps the assistant focused on software engineering, programming, computer science, and IT topics.
9. Read the assistant answer from the AI provider response.
10. Read token usage from the AI provider response when available.
11. If the assistant answer is empty, treat it as a server error.
12. Insert the assistant response into the database.

- `role` should be `"assistant"`.
- `content` should be the generated answer.
- `token_count` should be the token usage number, or `0` if not available.

13. Fetch the newly inserted user row and assistant row from the database.
14. Return both rows to the frontend.

Important implementation note:

- In this project, the previous history is loaded before inserting the new user question.
- The AI provider receives the previous history plus the new question.
- The visible chat history endpoint returns up to 100 rows, but the AI context uses only the latest 5 previous rows.

AI Assistant Behavior

The backend should guide the AI assistant with server-side instructions. The goal is to make this clone behave like a programming assistant for software engineering and IT-related questions.

The assistant should:

- Help with software engineering, programming, debugging, computer science, and IT questions.
- Explain concepts clearly and practically.
- Use Markdown when helpful.
- Wrap code examples in proper code blocks.

- Politely decline unrelated topics and guide the user back to programming.
- Avoid harmful, malicious, or unethical code.

These rules belong on the server because the frontend should not be trusted to control the assistant's safety or purpose.

Success Response

Status: `201 Created`

Field	Type	Description
`success`	`boolean`	`true` when the user and assistant rows are saved.
`message`	`string`	Success message.
`data`	`object`	Response payload wrapper.
`data.userConversation`	`object`	The saved user message.
`data.assistantConversation`	`object`	The saved assistant message.

`data.userConversation` and `data.assistantConversation` both use this shape:

Field	Type	Description
`id`	`number`	Message id from the database.
`role`	`string`	Either `"user"` or `"assistant"`.
`content`	`string`	Saved message text.
`tokenCount`	`number`	Token usage from the database. User messages usually return `0`.
`createdAt`	`string`	Time the message was created.

Example:

```
{
  "success": true,
  "message": "Conversation posted successfully.",
  "data": {
    "userConversation": {
      "id": 10,
      "role": "user",
      "content": "What is a closure in JavaScript?",
      "tokenCount": 0,
      "createdAt": "2026-05-10T12:00:00.000Z"
    },
    "assistantConversation": {
      "id": 11,
      "role": "assistant",
      "content": "A closure is a function that
      keeps access to variables from its outer scope
      after the outer function has finished running.",
      "tokenCount": 123,
      "createdAt": "2026-05-10T12:00:01.000Z"
    }
  }
}
```


Validation Error Responses

Status: `400 Bad Request`

Missing `question`:


```
{
  "status": false,
  "message": "question is required"
}
```


Empty `question`:


```
{
  "status": false,
  "message": "question is required"
}
```


Question is longer than `65535` characters:

```
{
  "status": false,
  "message": "question is too long"
}
```

Server Error Responses

Status: `500 Internal Server Error`

Missing AI provider API key:

```
{
  "status": false,
  "message": "AI provider API key is not configured."
}
```

Empty AI response:

```
{
  "status": false,
  "message": "The model returned an empty answer"
}
```

Generic server error:

```
{
  "status": false,
  "message": "Something went wrong try again later"
}
```

Happy coding 😀 .